

Soal 1: Candy Partition

Penjelasan

Diketahui terdapat n permen dan k anak. Berapa banyak cara **berbeda** pembagian permen jika setiap anak harus mendapatkan setidaknya satu permen.

- Jika banyak cara adalah bilangan **ganjil**, keluarkan `There are X ways to share the candies, A magical Halloween night!`
- Jika banyak cara adalah bilangan **genap**, keluarkan `There are X ways to share the candies, A spooky Halloween night!`
- Jika tidak ada cara agar semua anak mendapatkan minimal 1 permen, keluarkan `No fair way to share the candies tonight... A truly spooky Halloween!`

di mana X adalah banyak cara berbeda pembagian.

Solusi

Jika $n \geq k$, kita dapat membagikan 1 permen ke setiap anak sehingga tersisa $n - k$ permen saja. Selanjutnya, kita tinggal menghitung banyak cara membagikan $n - k$ permen kepada k anak. Namun, jika $n < k$, tidak ada cara agar setiap anak mendapatkan minimal 1 permen. Dengan menggunakan *stars and bars*.

$$\binom{(n - k) + k - 1}{k - 1} = \binom{n - 1}{k - 1}$$

Untuk menghitungnya, kita dapat gunakan rekursi dan identitas pascal. Misalkan $Comb(n, k)$ adalah fungsi untuk menghitung banyak kombinasi dari n yang dipilih sebanyak k .

$$Comb(n, k) = \begin{cases} 0, & n < k \\ 1, & n = k \text{ atau } k = 0 \\ Comb(n - 1, k - 1) + Comb(n - 1, k), & \text{lainnya} \end{cases}$$

Karena $1 \leq n, k \leq 30$ dan kompleksitas rekursi $O(2^n)$, sehingga perlu dilakukan optimasi sedikit untuk handle saat $n, k = 30$. Inisiasi array `memo[n][k]` yang akan digunakan untuk menyimpan hasil $Comb(n, k)$ yang sudah pernah dihitung agar tidak terjadi pemanggilan berulang. Sehingga kompleksitas kode menjadi $O(nk)$.

Berikut kode untuk soal di atas.

candy.c

```
#include <stdio.h>

int memo[31][31];

int comb(int n, int k){
    if(n == k || k == 0){
        return 1;
    }
}
```

```

    }

    if(memo[n][k] != 0){
        return memo[n][k];
    }
    memo[n][k] = comb(n - 1, k - 1) + comb(n - 1, k);
    return comb(n - 1, k - 1) + comb(n - 1, k);
}

int main(){
    int n, k;
    for(int i = 0; i < n; i++){
        for(int j = 0; j < k; j++){
            memo[i][j] = 0;
        }
    }
    scanf("%d %d", &n, &k);
    if(n < k){
        printf("No fair way to share the candies tonight... A truly spooky Halloween!");
        return 0;
    }
    int ans = comb(n - 1, k - 1);
    if(ans & 1){
        printf("There are %d ways to share the candies, A magical Halloween night!", ans);
    }else{
        printf("There are %d ways to share the candies, A spooky Halloween night!", ans);
    }
}

```

Soal 2: GCD & LCM

Penjelasan

Implementasikan fungsi *GCD* (*Greatest Common Divisor*) untuk menghitung fungsi *LCM* (*Least Common Multiple*) dari dua buah bilangan menggunakan fungsi rekursif. Diberikan rumus *GCD* yaitu,

$$GCD(n, m) = \begin{cases} n, & m = 0 \\ GCD(m, n \% m), & m \neq 0 \end{cases}$$

dan rumus *LCM* yaitu,

$$LCM(n, m) = \frac{|n \cdot m|}{GCD(n, m)}$$

Solusi

Kita cukup mengimplementasikan fungsi sesuai rumus pada soal. Untuk menghindari perkalian $n \cdot m$ yang sangat besar saat $n, m = 10^9$, kita akan lakukan pembagian antara n dan $GCD(n, m)$ terlebih dahulu lalu kalikan dengan m untuk menghindari *overflow*.

Berikut kode untuk soal di atas.

lcm.c

```
#include <stdio.h>

long long gcd(long long a, long long b){
    if(b == 0){
        return a;
    }
    return gcd(b, a % b);
}

long long lcm(long long a, long long b){
    return a / (gcd(a,b)) * b;
}

int main(){
    long long a, b;
    scanf("%lld %lld", &a, &b);
    long long ans = lcm(a,b);
    printf("%lld", ans);
}
```

Soal 3: When yah

Penjelasan

Diberikan sebuah grid $R \times C$ di mana setiap sel mewakili **daratan kering** (.), **air** (0), dan **harta karun** (1–9). Air dan harta karun merupakan bagian dari "**danau**". Sebuah "**danau**" didefinisikan sebagai sekumpulan sel air (0) dan **harta karun** (1 – 9) yang terhubung secara empat arah (atas, bawah, kiri, kanan). Sel **daratan** (.) bertindak sebagai pemisah antardanau. Tentukan **nilai total maksimum** harta karun yang bisa diperoleh pada grid dari satu "**danau**" saja.

Solusi

Untuk setiap sel yang merupakan bagian dari danau (0 – 9) dan belum dikunjungi, lakukan penjelajahan secara empat arah (atas, bawah, kiri, kanan) secara rekursif untuk menemukan seluruh bagian danau. Pada saat penjelajahan,

1. Tandai setiap sel yang sudah dikunjungi agar tidak terjadi *stack overflow*.
2. Jika bertemu angka (0 – 9) konversikan ke integer lalu dijumlahlahkan.
3. Pastikan penjelajahan dilakukan di dalam grid ($0 \leq x < n$ dan $0 \leq y < m$) untuk menghindari pengaksesan di luar memori array grid.
4. Jika total harta karun sudah diperoleh untuk satu danau, ulangi penjelajahan untuk semua danau dan ambil nilai maksimum dari semuanya.

Berikut kode untuk soal di atas.

dfs.c

```
#include <stdio.h>

int visited[101][101];
```

```

char grid[101][101];
int sum;

int to_num(char c){
    return (int)(c - 48);
}

int isvalid(int x, int y, int n, int m){           // Cek masih dalam grid dan cell
selanjutnya belum dikunjungi
    return (x >= 0 && y >= 0 && x < n && y < m && !visited[x][y]);
}

void dfs(int x, int y, int n, int m){
    visited[x][y] = 1;                           // Tandai udah dikunjungi
    int num = to_num(grid[x][y]);                 // Ubah char ke integer
    sum += (num);
    if(grid[x + 1][y] != '.' && isvalid(x + 1, y, n, m)){ // Kanan
        dfs(x + 1, y, n, m);
    }
    if(grid[x - 1][y] != '.' && isvalid(x - 1, y, n, m)){ // Kiri
        dfs(x - 1, y, n, m);
    }
    if(grid[x][y + 1] != '.' && isvalid(x, y + 1, n, m)){ // Bawah
        dfs(x, y + 1, n, m);
    }
    if(grid[x][y - 1] != '.' && isvalid(x, y - 1, n, m)){ // Atas
        dfs(x, y - 1, n, m);
    }
}

int main(){
    int n, m;
    scanf("%d %d", &n, &m);
    for(int i = 0; i < n; i++){
        char temp[101];
        scanf("%s", temp);
        for(int j = 0; j < m; j++){
            visited[i][j] = 0;
            grid[i][j] = temp[j];
        }
    }
    int mx = 0;
    for(int i = 0; i < n; i++){
        for(int j = 0; j < m; j++){
            if(grid[i][j] != '.' && !visited[i][j]){
                sum = 0;
                dfs(i, j, n, m);
                if(mx < sum){
                    mx = sum;
                }
            }
        }
    }
    printf("%d", mx);
}

```

Soal 4: Banyak Banget Cangkang

Penjelasan

Tentukan nilai dari cangkang yang ditemukan dengan ketebalan b menggunakan kalkulator yang memiliki batas kedalaman rekursi n dengan fungsi berikut.

$$S(b, n) = b + \frac{b}{b + \frac{b}{b + \dots}}$$

Jika fungsi telah mencapai kedalaman rekursi, maka fungsi hanya mengembalikan angka b saja. Keluarkan nilai cangkang dengan presisi 8 digit di belakang koma.

Solusi

Berdasarkan pola dari keterangan soal, kita mendapatkan rumus rekursi yaitu,

$$S(b, n) = \begin{cases} b, & n = 0 \\ b + \frac{b}{S(b, n-1)}, & n \neq 0 \end{cases}$$

Jika $1 \leq n \leq 10^{12}$, maka akan terjadi *stack overflow* karena fungsi yang dipanggil terlalu banyak. Karena soal hanya meminta untuk mengeluarkan output dengan presisi 8 digit setelah koma, kita dapat membatasi nilai n saat $n = 20$.

b	n	$S(b, n)$
1	18	1,61803396
1	19	1,61803400
1	20	1,61803399
1	21	1,61803399

Dapat dicek bahwa saat $n \geq 20$ nilai fungsi tidak berubah. Sehingga untuk $n \geq 20$, kita langsung mengeluarkan $S(b, 20)$. *Otherwise*, jalankan fungsi sebagaimana mestinya.

Berikut kode untuk soal di atas.

cangkang.c

```
#include <stdio.h>

double fun(long long b, long long n){
    if(n == 0){
        return b;
    }
    return b + (b * 1.0) / (fun(b, n - 1));
}

int main(){
    long long b, n;
    scanf("%lld %lld", &b, &n);
    double ans;
    if(n >= 20){
        ans = fun(b, 20);
    }else{

```

```
        ans = fun(b, n);  
    }  
    printf("%.8f", ans);  
}
```